

Online Judge Platform

Prepared By: Kuswanth Tumma

System Diagrams Package

Online Judge Platform — Architectural & Workflow Visualizations

| Document Metadata | Details |

| :--- | :--- |

| **Project Title** | Online Judge Platform |

| **Diagram Engine** | Native Mermaid JS Syntax |

| **Document Version** | 1.0.0-RELEASE |

| **Status** | Approved for Implementation |

| **Author** | Kuswanth Tumma |

| **Date** | July 2026 |

1. High-Level System Architecture Diagram

```

graph TD
    Client[React 18 SPA + Monaco Editor] -->|HTTPS REST / WSS| Nginx[Nginx Reverse Proxy & Load Balancer]
    Nginx -->|Proxy Pass| Express1[Express API Node 1]
    Nginx -->|Proxy Pass| Express2[Express API Node 2]

    Express1 -->|Read / Write| Mongo[(MongoDB Replica Set)]
    Express2 -->|Read / Write| Mongo

    Express1 -->|Enqueue Submission| Redis[(Redis + BullMQ Queue)]
    Express2 -->|Enqueue Submission| Redis

    Redis -->|Dequeue Job| Worker1[BullMQ Evaluation Worker 1]
    Redis -->|Dequeue Job| Worker2[BullMQ Evaluation Worker 2]

    Worker1 -->|Fetch Testcases| Mongo
    Worker2 -->|Fetch Testcases| Mongo

    Worker1 -->|Spawn Sandbox| Docker1[Ephemeral Docker Sandbox C++]
    Worker2 -->|Spawn Sandbox| Docker2[Ephemeral Docker Sandbox Python]

    Docker1 -->|Verdict & Metrics| Worker1
    Docker2 -->|Verdict & Metrics| Worker2

    Worker1 -->|Update Submission Status| Mongo
    Worker2 -->|Update Submission Status| Mongo

```

2. System Use Case Diagram

```
graph LR
    subgraph Users
        Candidate[Regular User]
        AdminUser[System Administrator]
    end

    subgraph Online_Judge_Platform_System_Boundary [Online Judge Platform System Boundary]
        UC1(Register Account)
        UC2(Login & Receive JWT)
        UC3(View & Search Problems)
        UC4(Submit Code Solution)
        UC5(View Real-Time Verdict)
        UC6(View Submission History)
        UC7(View Leaderboard)
        UC8(Create / Edit Problem)
        UC9(Upload Hidden Testcases)
        UC10(Manage User Roles)
    end

    Candidate --> UC1
    Candidate --> UC2
    Candidate --> UC3
    Candidate --> UC4
    Candidate --> UC5
    Candidate --> UC6
    Candidate --> UC7

    AdminUser --> UC2
    AdminUser --> UC8
    AdminUser --> UC9
    AdminUser --> UC10
```

3. UML Class Diagram

```
classDiagram
class User {
    +ObjectId id
    +String name
    +String email
    +String passwordHash
    +UserRole role
    +Int problemsSolved
    +Int totalSubmissions
    +register()
    +login()
}

class Problem {
    +ObjectId id
    +String title
    +String slug
    +String statement
    +DifficultyLevel difficulty
    +Object constraints
    +String sampleInput
    +String sampleOutput
    +createProblem()
    +updateConstraints()
}

class TestCase {
    +ObjectId id
    +ObjectId problemId
    +String input
    +String expectedOutput
    +Boolean isHidden
    +Int order
}

class Submission {
    +ObjectId id
    +ObjectId userId
    +ObjectId problemId
    +String language
    +String sourceCode
    +VerdictType verdict
    +Int executionTimeMs
    +Int memoryUsedKb
    +evaluate()
}

class DockerExecutionService {
    +spawnContainer()
    +compileSource()
}
```

```

    +runTestcase()
    +harvestMetrics()
  }

  User "1" -- "*" Submission : submits
  Problem "1" -- "*" TestCase : contains
  Problem "1" -- "*" Submission : evaluated_by
  Submission ..> DockerExecutionService : processed_by

```

4. System Component Diagram

```

componentDiagram
    package "Presentation Layer" {
        [React Client Component]
        [Monaco Code Editor]
    }

    package "API Gateway Layer" {
        [Authentication Controller]
        [Problem Controller]
        [Submission Controller]
        [JWT Middleware]
    }

    package "Async Messaging Layer" {
        [Redis Instance]
        [BullMQ Submission Queue]
    }

    package "Worker Compute Layer" {
        [BullMQ Worker Loop]
        [Docker Orchestrator Service]
        [Diff Evaluator Engine]
    }

    package "Storage Layer" {
        database "MongoDB Cluster"
    }

    [React Client Component] --> [JWT Middleware]
    [JWT Middleware] --> [Submission Controller]
    [Submission Controller] --> [BullMQ Submission Queue]
    [BullMQ Submission Queue] --> [BullMQ Worker Loop]
    [BullMQ Worker Loop] --> [Docker Orchestrator Service]
    [Docker Orchestrator Service] --> [Diff Evaluator Engine]
    [Diff Evaluator Engine] --> [MongoDB Cluster]

```

5. Production Infrastructure Deployment Diagram

```

graph TB
    subgraph Cloud_Infrastructure [Cloud Infrastructure AWS / GCP]
        subgraph Public_Subnet [Public Subnet]
            ALB[AWS Application Load Balancer]
        end

        subgraph Private_App_Subnet [Private App Subnet]
            API1[Express API Container 1]
            API2[Express API Container 2]
            Worker1[BullMQ Evaluation Worker 1]
            Worker2[BullMQ Evaluation Worker 2]
        end

        subgraph Private_Data_Subnet [Private Data Subnet]
            Redis[(Redis Enterprise Cluster)]
            MongoPrimary[(MongoDB Primary)]
            MongoSec1[(MongoDB Secondary 1)]
            MongoSec2[(MongoDB Secondary 2)]
        end
    end

    Internet[Internet Clients] -->|HTTPS Port 443| ALB
    ALB -->|HTTP Port 5000| API1
    ALB -->|HTTP Port 5000| API2

    API1 --> Redis
    API2 --> Redis

    Worker1 --> Redis
    Worker2 --> Redis

    API1 --> MongoPrimary
    API2 --> MongoPrimary
    Worker1 --> MongoPrimary
    Worker2 --> MongoPrimary

    MongoPrimary -.->|Replication| MongoSec1
    MongoPrimary -.->|Replication| MongoSec2

```

6. Code Submission & Async Verdict Sequence Diagram

```
sequenceDiagram
    autonumber
    actor User as User Browser
    participant API as Express API
    participant DB as MongoDB
    participant Queue as Redis (BullMQ)
    participant Worker as Evaluation Worker
    participant Docker as Docker Sandbox

    User->>API: POST /api/v1/submissions (code, lang, problemId)
    API->>API: Validate Request & JWT
    API->>DB: Save Submission (Status: "Pending")
    DB-->>API: Return submissionId
    API->>Queue: Push Job {submissionId, problemId, lang, code}
    API-->>User: 202 Accepted {submissionId, status: "Pending"}

    Queue->>Worker: Dequeue Job Payload
    Worker->>DB: Update Status ("Processing")
    Worker->>DB: Fetch Hidden Testcases for problemId
    DB-->>Worker: Return Array of Testcases

    Worker->>Docker: Spawn Sandbox Container (cgroups, seccomp, no-net)
    activate Docker
    Docker->>Docker: Compile Code (if C++/Java)
    loop For Each Testcase
        Docker->>Docker: Execute Program with Input
        Docker-->>Worker: Return stdout, stderr, exitCode, time, memory
        Worker->>Worker: Compare stdout vs expectedOutput (DiffEvaluator)
    end
    deactivate Docker

    Worker->>DB: Save Final Verdict ("Accepted", timeMs, memoryKb)
    Worker->>Queue: Complete Job

    loop Polling Status
        User->>API: GET /api/v1/submissions/:submissionId
        API->>DB: Query Submission Record
        DB-->>API: Return Submission Record
        API-->>User: 200 OK {verdict: "Accepted", executionTimeMs: 14}
    end
```

7. User Authentication & JWT Lifecycle Sequence Diagram

```
sequenceDiagram
    autonumber
    actor Client as React Client
    participant Router as Express Auth Router
    participant Service as Auth Service
    participant DB as MongoDB

    Client->>Router: POST /api/v1/auth/login {email, password}
    Router->>Service: authenticateUser(email, password)
    Service->>DB: findOne({email})
    DB-->>Service: User Document with passwordHash
    Service->>Service: bcrypt.compare(password, passwordHash)

    alt Invalid Credentials
        Service-->>Router: Throw AuthError
        Router-->>Client: 401 Unauthorized {error: "Invalid credentials"}
    else Valid Credentials
        Service->>Service: jwt.sign({userId, role}, secret, {expiresIn: '24h'})
        Service->>Router: {token, userPayload}
        Router-->>Client: 200 OK {token: "eyJhbG...", user: {...}}
    end

    Note over Client, Router: Subsequent Protected API Request
    Client->>Router: GET /api/v1/problems (Header: Bearer eyJhbG...)
    Router->>Router: authMiddleware() verifying JWT Signature
    Router-->>Client: 200 OK {data: [problems]}
```

8. Code Submission Activity & Evaluation Lifecycle

```
stateDiagram-v2
    [*] --> Submitted: User clicks Submit

    state Submitted {
        [*] --> Enqueued: Save to DB & Enqueue to Redis
        Enqueued --> Processing: Dequeued by BullMQ Worker
    }

    state Processing {
        [*] --> Compiling: Language = C++ / Java
        Compiling --> CompilationError: Build fails
        Compiling --> Executing: Build succeeds

        [*] --> Executing: Language = Python

        state Executing {
            [*] --> RunningTestcase
            RunningTestcase --> TimeLimitExceeded: time > timeLimit
            RunningTestcase --> MemoryLimitExceeded: memory > memoryLimit
            RunningTestcase --> RuntimeError: exitCode != 0
            RunningTestcase --> Diffing: Output captured
            Diffing --> WrongAnswer: Output mismatch
            Diffing --> NextTestcase: Output matched
            NextTestcase --> RunningTestcase: More testcases remain
            NextTestcase --> Accepted: All testcases passed
        }
    }

    CompilationError --> FinalVerdict
    TimeLimitExceeded --> FinalVerdict
    MemoryLimitExceeded --> FinalVerdict
    RuntimeError --> FinalVerdict
    WrongAnswer --> FinalVerdict
    Accepted --> FinalVerdict

    FinalVerdict --> [*]: Update DB & Notify User
```

9. Role-Based Access Control (RBAC) Flowchart

```
flowchart TD
```

```

    Start([Incoming HTTP Request]) --> CheckAuth{Is Bearer JWT Token Present?}
    CheckAuth -- No --> Reject401[Return 401 Unauthorized]
    CheckAuth -- Yes --> VerifyJWT{Is JWT Valid & Unexpired?}
    VerifyJWT -- Invalid --> Reject401
    VerifyJWT -- Valid --> ExtractRole[Extract User Role from Token]

    ExtractRole --> CheckRoute{Is Endpoint Admin-Only?}
    CheckRoute -- No --> AllowUser[Allow Access: Proceed to Controller]
    CheckRoute -- Yes --> CheckAdmin{Is Role == 'Admin'?}
    CheckAdmin -- No --> Reject403[Return 403 Forbidden: Insufficient Rights]
    CheckAdmin -- Yes --> AllowAdmin[Allow Access: Proceed to Admin Controller]

```

10. Docker Sandbox Security & Isolation Architecture

```
graph TD
```

```

    subgraph Host_System_Environment
        HostKernel[Linux OS Kernel v6.x]

        subgraph Docker_Daemon_Security_Boundary
            subgraph Container_Isolation_Boundary
                Proc[User Code Execution Process UID 10001]
                CGroups[Linux cgroups v2: RAM 256MB / CPU 1.0 / PIDs 64]
                Seccomp[Seccomp Syscall Filter: Block sys_execve / sys_socket]
                Net[Network Namespace: Unlinked --network none]
                Mount[Mount Namespace: Read-Only Volume /sandbox:ro]
            end
        end

        HostKernel --- CGroups
        HostKernel --- Seccomp
        HostKernel --- Net
        HostKernel --- Mount

        Proc --> CGroups
        Proc --> Seccomp
        Proc --> Net
        Proc --> Mount
    end

```

11. CI/CD Deployment Pipeline Flowchart

flowchart LR

Dev([Developer Push Code]) --> GitHub[GitHub Repository]

GitHub --> Action[Trigger GitHub Actions]

subgraph CI Pipeline Execution

Action --> Lint[ESLint & Prettier Checks]

Lint --> Test[Run Jest Unit & Integration Tests]

Test --> Security[Run Security Audit & Vulnerability Scan]

end

Security --> Build[Build Production Docker Images]

Build --> ECR[Push to Container Registry]

subgraph CD Continuous Deployment

ECR --> K8s[Kubernetes Cluster Rolling Upgrade]

K8s --> HealthCheck{Health Check Probe OK?}

HealthCheck -- Failed --> Rollback[Trigger Automated Rollback]

HealthCheck -- Success --> Finish([Deployment Live])

end